Accelerated Computing Analysis of The Lebwohl-Lasher Model

School of Chemistry, University of Bristol
Rose Fernandes

Abstract

This study presents a critical analysis of accelerating a Python implementation of a 2D Lebwohl-Lasher liquid crystal model. Four techniques were implemented and tested: Cython with OpenMP, distributed memory parallelisation using mpi4py, NumPy vectorisation, and Numba JIT compilation. Performance was assessed across various lattice sizes and varied numbers of threads or workers. Ultimately, Numba JIT compilation achieved the best speed-ups, closely followed by Cython with OpenMP. NumPy achieved significant speed-ups with minimal changes required to the code and mpi4py was the most challenging to implement with the weakest speed-ups. The different methods were compared based on their ease of implementation, flexibility and maintenance required.

Introduction

Accelerated computing is an important consideration for running large models and can provide significant speedups. This is critical to avoid consuming large amounts of computer power, which may have vast environmental impacts, especially when using high-performance computing techniques. Furthermore, it can run larger calculations faster, which is convenient for programs that are repeatedly run, also ensuring a better user-experience. There are many techniques available to do this, from small changes to the way code is executed or completely re-writing scripts to optimise them for certain techniques, a variety of which will be researched and tested in this study.[1]

These techniques can be carried across many languages from compiled languages (C or C++) to non-compiled languages (Python). Compiled languages are typically faster so an important area of focus is accelerating Python scripts. This is a very popular coding language, so enabling speedups here would be very useful for a wide range of people.

A wide range of techniques used here, including: mpi4py, cython, NumPy vectorization, and Numba. The details of which will be described in the following background section. These methods will be implemented, and each method will be compared to on the following criteria: the ease of use and implementation, the level of speedup acquired, the computational power required for each method and how useful this speed up is.

To allow a fait comparison of each technique a single script describing the Lebwohl-Lasher model for liquid crystals will be accelerated.[2] This will act as our benchmark for the study. This model was chosen as there are a variety of functions within the code that could be individually sped up. Due to the implementation of a Metropolis Monte-Carlo technique[3] and lattice simulation, which are widely used within the scientific programming community, this piece of code will be representative of speed-ups available across scientific computing.

Background

Mpi4py

Mpi4py implements distributed programming where the lattice was partitioned and halo exchanges were used to communicate neighbouring changes.

Cython

Cython compiles code in C, which is an inherently faster language than python. Typed memory views, and C-libraries were used to accelerate the code. Parallel implementations were completed using a checkerboard system to avoid race conditions.

NumPy Vectorization

NumPy vector replaced many nested loops, as nested loops tend to be quite slow in python.

Numba

Numba compiles Python into machine code via LLVM at runtime. Functions were decorated with @jit(nopython=True) for fastest compilation with parallel=True for parallel parts. A checkerboard system was used for parallel parts to avoid race conditions.

Model

The model is initialised by splitting space into cells and giving each cell an average orientation. This is named a liquid crystal director and each cells interaction with neighbouring cell is defined by Equation 1. This calculation is conducted within the one_energy function.

$$u_j = -\frac{\in}{2}\sum_{i=1}^{n}(3cos^2\left(\sigma_j - \sigma_i\right) - 1) \tag{1}$$

n = n nearest neighbours
ϵ = maximum interaction energy between pairs of adjacent crystal directors
σ_i = average orientation of cell i

σ_j = average orientation of cell j
u_j = potential energy of the cell

Once all the energies have been computed these are used to calculate energy of the whole lattice in the function all_energy.

A Metropolis Monte-Carlo technique was used for the simulation. It ran as such:

1. A random cell was picked
2. The energy of that cell was calculated
3. A random change was made to the angle
4. The new energy and energy difference was calculated
5. The energy change was immediately accepted if the new energy was less than or equal to the old energy
6. Otherwise, a random number would be generated, in the range 0.0-1.0, and if this was less than the Boltzmann factor the change would be excepted
7. Otherwise, the change would be rejected

This process was repeated until every cell in the lattice was visited on average once. This sweep was then repeated for many Monte Carlo steps.

Finally, the orientational order was calculated using Equation 2.

$$Q_{\alpha\beta} = \frac{1}{2} < 3l_{\alpha,i} l_{\beta,i} - \delta_{\alpha\beta} >_i \quad \alpha, \beta = x, y, z \tag{2}$$

Computational Details

The code archive can be found here: https://github.com/rosefernandesx/lebwohl-lasher/tree/main. Instructions to run each file are within the description of each script.

To ensure fair tests 50 Monte-Carlo steps were used for each run with 0 plots being produced and temperature dependence set to 0.5. Runtimes were averaged over three runs. All tests were run on a MacBook Pro with an Apple M4 chip. This had four fast cores and 6 slower ones which may have affected the performance when more than 4 cores were used.

Results files were assessed to check they aligned with the original code output, to ensure the new techniques still carried out the correct calculations.

Results and Discussion

Initially, the original code ("Lebwohl_Lasher.py") was profiled, for a 50x50 lattice with 50 Monte-Carlo steps, using the python profiler (ran "python -m cProfile LebwohlLasher.py 50 50 0.5 0 > prof.txt" in the command line). Table 1 presents, that the function calculating the energy for each cell (one_energy), using Equation 1, took up the most time (37.7 % of the run time at 0.592 s for a 1.570 s run) as it is called by the two other functions (all_energy and MC_step). Therefore, this was the first targeted function for accelerating the code. Following this, the MC step, get_order and all_energy. Therefore, these functions will be the focus for the speed up too. It is important to note that the functions initdat, plotdat, savedat were each only called once and took negligible time, so accelerating these parts of the code would be redundant. The main function also followed a similar pattern, but its cumulative time was much greater than the other single-call functions since it spent most of its time inside other functions. Therefore, these functions will be sped up since they will cause the greatest difference.

Table 1 - Summary of the number of calls and cumulative time taken for the slowest functions in the original Lebhowl-Lasher program for a 50x50 lattice with 50 Monte-Carlo steps. The total time taken was 1.570 s, and percentages overlap as some functions are nested within others.

Function	Number of calls	Time taken	Percentage of overall time
one_energy	37750	0.592	37.7%
MC_step	50	0.574	36.6%
get_order	51	0.339	21.6%
all_energy	51	0.190	12.1%

The first attempt for accelerating the code was using Cython with OpenMP. Dynamic scheduling was used to prevent one core waiting on other cores. Cython yielded significant speed up, especially over larger lattice sizes, as seen in Figure 1.

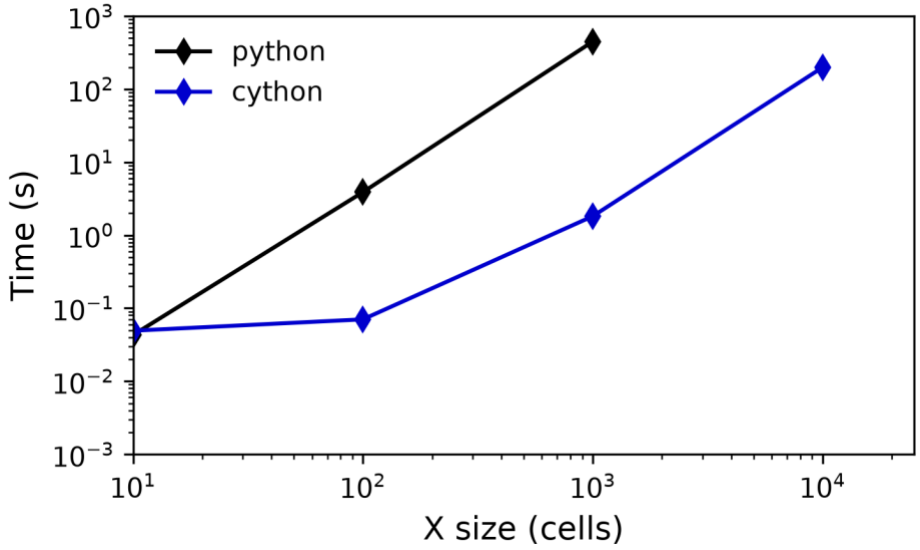


Figure 1 - Comparison of raw python code with parallel cython code across varying lattice sizes for 50 Monte-Carlo steps with T=0.5.

This technique gave very decent speed-ups and is suitable for many levels of programming ability. For example, raw python scripts can be cythonized, with no changes required to the code. This is easily built up using typed memory views and C definitions of variables. However, the parallel element is more challenging and time-consuming to implement as an appropriate technique, here the checkerboard technique, needs to be used to avoid race conditions. Furthermore, it isn't obvious when these occurs as no error is raised by the program. In terms of flexibility, cython provides complete control over which and how many loops you parallelise. Furthermore, C-level code and high-level python code can co-exist within the same script, so python plotting tools can be used alongside C compilation. However, this is limited by the fact that this is shared memory on a single node, so separate nodes cannot be used, as done in MPI. Also, the use of OpenMP is more limited within python than when implemented with C/C++. Furthermore, only the cores on your own laptop can be used, so the speed-up depends on your own device which can be limiting. Finally, only CPU time can be used and not GPUs, which can be faster. Regarding maintenance, the program is contained within one script and is relatively compact. However, setup and run scripts are also required. Setup scrips are device specific and would need to be changed when sharing code with others who use a different operating system. New users would also need the OpenMP, and other dependencies installed for the script to run. This technique could be improved by further parallelising the code, for example by combing some functions, so calculating the total energy and order of the lattice could be conducted within a single sweep.

Next, mpi4py methods were implemented and tested with carrying numbers of workers, with a comparison to the original python code in Figure 2.

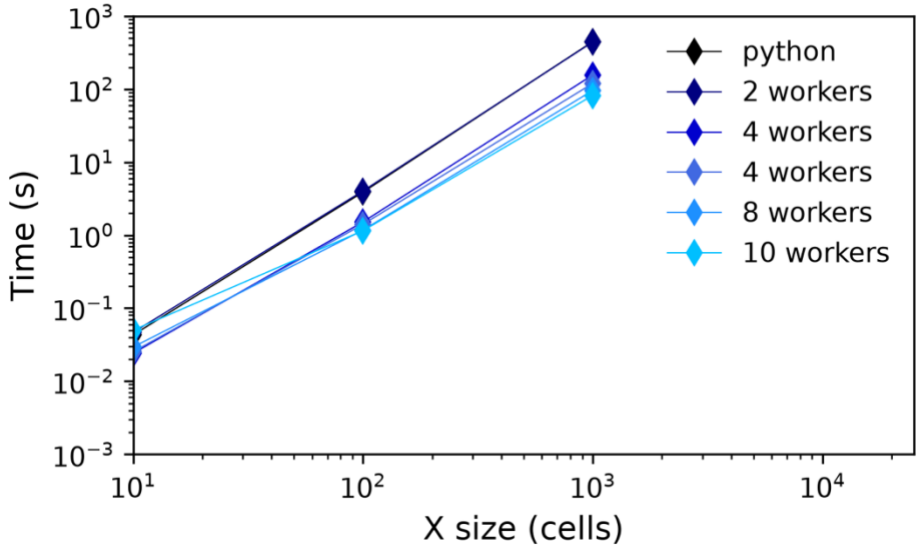


Figure 2 - Comparison of raw python code with mpi4py accelerated code with varying numbers of workers across varying lattice sizes for 50 Monte-Carlo steps with T=0.5.

This technique provided moderate speed-ups. The speed-ups improved as more workers were used, except when a larger number of workers were used for smaller lattices due to the overhead in communication outweighing the time taken for the calculations required. This was the most challenging technique to implement due to its distributed-memory model, with much care required for accurate send and receive messages. More of the script had to be restructured with the lattice being partitioned and point-to-point communication and halo exchanges occurring. Therefore, this technique is only suitable to those with more advanced python skills. Mpi4py has excellent flexibility as it can be used on High-Performance Computing clusters, which would likely yield greater speed ups than a single laptops memory limits. However, this technique required hard coded send and receive messages which would be a great barrier if changes to the code's functionality were required. These messages may need to be recomputed. There is also a strong dependence on the underlying OpenMPI framework, so the technique only remains viable as long as this framework is still maintained. Finally, it also remains limited to CPU execution and cannot leverage GPU execution. Maintaining the code is also a challenge as the communication calls are interwoven within the script and would be very difficult to change, especially when working collaboratively. It would also be more difficult for a new user to test the code as MPI libraries must be installed. This technique could be improved by optimizing halo exchange frequency.

NumPy vectorization was then tested, and the results are shown in Figure 3.

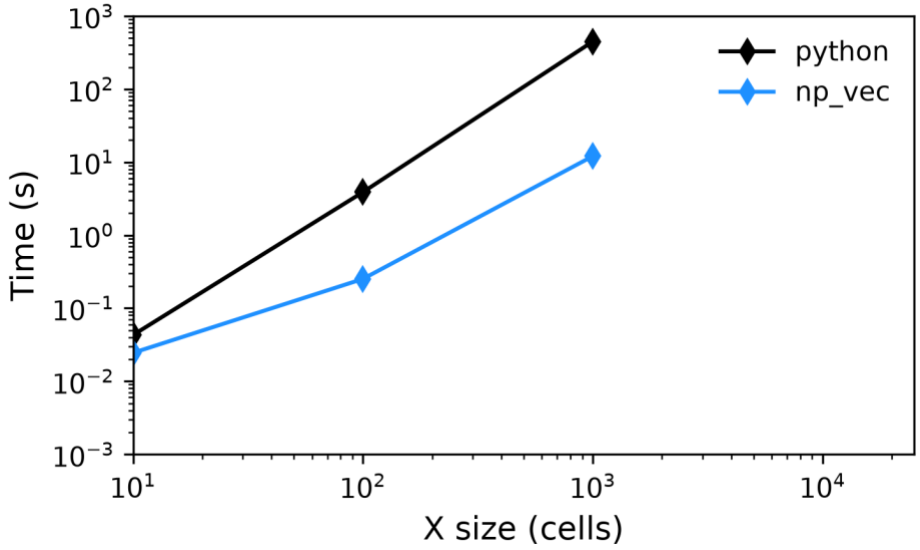


Figure 3 - Comparison of raw python code with NumPy vectorized code across varying lattice sizes for 50 Monte-Carlo steps with T=0.5.

The vectorized NumPy technique provided generous speed-ups and is well-suited for minimal effort speed-ups. Nested loops were replaced with more efficient array operations. This simple change allows a lot of time to be saved as nested loops are extremely time-consuming in Python. NumPy is very flexible as the script remains as clean, readable Python syntax and would be easy for someone new to pick it up and make changes without having to re-write the acceleration portions. Though, it does remain limited to single-node and CPU execution. For maintenance, the script is compact and requires the least additional modules to run. Therefore, it is easy to maintain and share. It can be considered a good practice for writing python code. This technique could be improved by combining with other acceleration methods and harness the speed-up of two combined techniques.

Finally, numba techniques were explored for accelerating the program. This technique provided excellent speed-ups, as illustrated in Figure 4. Numba can be used by a range of skill levels as it can be implemented simply as a compiler with function decorators and not much changes to the code. However, there are vast opportunities to extend by adapting the code for parallel techniques of use of GPU. The LLVM compilation is very flexible and supports vectorisation, multi-threading and use of GPUs. However, it is limited to single node runs and cannot be scaled across distributed clusters. It can be simply maintained as it is a single file with no run or setup scripts. However, it can be tricky to set up initially as many compilation errors will be faced. The implementation here could be improved by leveraging GPU access, however this is unavailable on MAC. This could be done on online platforms but would provide an unfair comparison to other techniques.

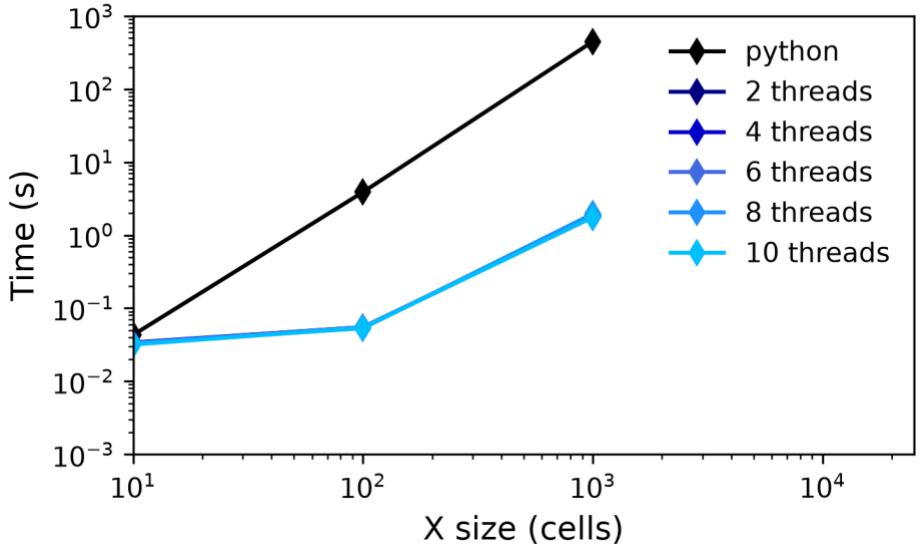


Figure 4 - Comparison of raw python code with numba accelerated code across with varying numbers of threads across varying lattice sizes for 50 Monte-Carlo steps with T=0.5.

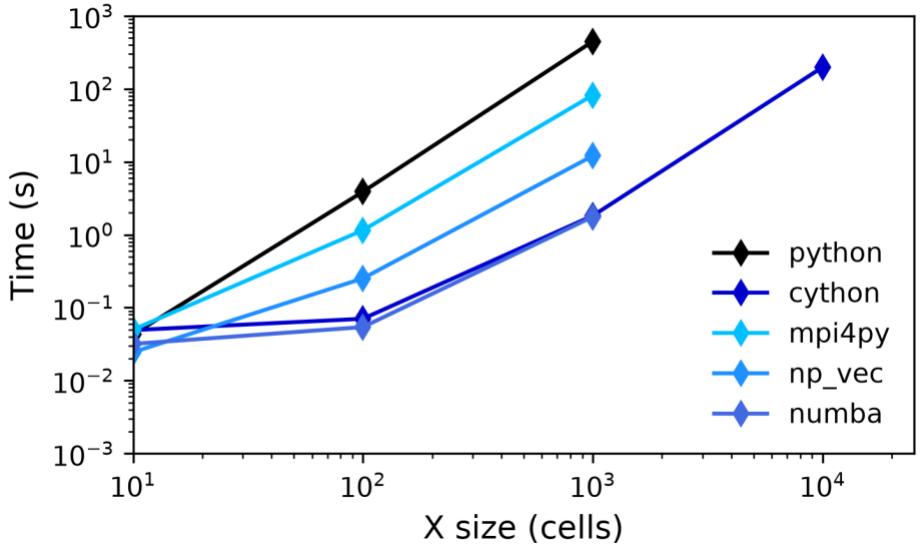


Figure 5 - Comparison of all accelerated techniques with the raw python code across varying lattice sizes for 50 Monte-Carlo steps with T=0.5. The fastest instances of mpi4py and numba were used with 10 workers and 10 threads respectively.

Figure 5 compares all the techniques and shows that numba performed the best closely followed by cython. The next best was NumPy vectorisation followed by mpi4py. One would conclude that mpi4py was the most difficult to implement for the least speed-up so would not be recommended, unless you wish to run it on a HPC system and see what speed-ups are possible there. NumPy vectorization was by far the easiest to implement and provided a generous speed up, so this is recommended for anyone who wants to accelerate their code

with minimal effort. Finally, the parallelisation techniques with cython and numba, using the checkerboard technique, allowed the best speed-ups but were quite tricky to implement, often requiring reworking of the original code. So, to achieve optimal acceleration a parallel technique should be used.

Conclusions

In conclusion, each technique was implemented and assessed in terms of speed-up acquired, ease of coding and maintenance, and the flexibility of each approach. Overall, numba achieved the best speed up, vectorized NumPy was the easiest to implement and mpi4py was the most flexible approach. Therefore, I would recommend vectorized NumPy or simple methods with cython or numba to a beginner programmer who is interested in experimenting with speeding up their code with minimal effort. Parallel techniques would be recommended to a more experienced programmer with more time on their hands to spend accelerating their code. And finally, mpi4py would be recommended to a programmer who has access to HPC systems and would like to utilise these to best speed up their code.

References

1. L. Lannelongue, J. Grealey, M. Inouye, Green Algorithms: Quantifying the Carbon Footprint of Computation. Adv. Sci. 2021, 8, 2100707. https://doi.org/10.1002/advs.202100707
2. Lebwohl, P.A. and Lasher, G., “Nematic-Liquid-Crystal Order—A Monte Carlo Calculation”, Phys. Rev. A, 6, 426–429 (1972).
3. N. Metropolis, A.W. Rosenbluth, M.N. Rosenbluth, A.H. Teller and E. Teller, J. Chem. Phys. 21, 1087 (1953).